Conteggio dei Bi-Grams e Tri-Grams

Mattia Baroncelli
7166468
mattia.baroncelli@edu.unifi.it

Abstract

Il progetto si concentra sull'implementazione di un algoritmo che consenta di contare le istanze di bi-gram e tri-gram all'interno di un testo. Questo inteso sia come composizione di caratteri, che come composizione di parole. E' chiaro, a partire dalla natura del progetto, come all'aumentare della dimensione del file dato in input, i tempi di esecuzione si dilatano notevolemnte. Lo scopo principale del progetto pertanto è quella di implementare una versione parallela dell'algoritmo, che vada a velocizzare il processo. La soluzione verrò data sia tramite la libreria Multiprocessing che utilizzando JobLib, valutando vantaggi e svantaggi dei due metodi.

Future Distribution Permission
The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

L'obiettivo principale del progetto è quello di creare un algoritmo che ci permetta di contare gli n-grams presenti in un file dato in input. Gli n-grams sono sequenze di n caratteri (o parole) consecutive che si riscontrano all'interno del testo. In particolare in questo progetto ci concentreremo sulla ricerca dei bi-gram e dei tri-gram (ossia le sequenze con n=2 e n=3). Nel caso dei bi-gram quindi l'algoritmo andrà a ricercare tutte le coppie di caratteri vicine all'interno del testo e successivamente conterà le occorrenze delle coppie all'interno del testo. Una delle maggiori criticità dell'algoritmo si presenta quando la dimensione del testo risulta essere molto grande, poichè il processo di contare le occorrenze all'interno del testo può durare molto tempo. Con lo scopo di superare queste limitazioni abbiamo implementato alcune versioni parallele dell'algoritmo, che riescono a suddividere le operazioni fra i vari core, andando a ridurre i tempi di esecuzione. Il processo di parallelizzazione è stato fatto usando Python; in particolare le due librerie *Multiprocessing* e *JobLib*. Andremo quindi a valutare quanto l'impatto della parallelizzazione migliori le prestazioni del conteggio degli n-grams.

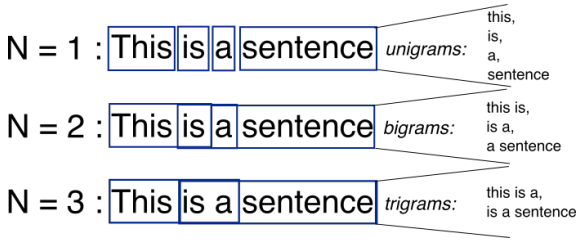

Figura 1. Esempio di uni-gram, bi-gram e tri-gram.

2. Implementazione

Andiamo adesso a vedere alcune fasi necessarie e scelte di implementazione del codice.

2.1. Dataset

Prima di iniziare a descrivere l'algoritmo è necessario procedere con una fase pregressa, ossia quella di normalizzazione del testo. Innanzitutto tutti i testi che andremo ad usare saranno in lingua inglese e sono estratti da testi provenienti da *Gutenberg Project*, un sito che permette l'accesso gratuito a numerosi testi della letteratura inglese in formato UTF-8. Un'ecccezione è rappresentata dal dataset *text8 dataset* che è una collezione della dimensione di 100 MB, di testo estratto da Wikipedia. Per poter avere a disposizione un file di dimensione a piacere abbiamo creato uno script `create_large_corpus` che permette di copiare il testo contenuto in un file per un numero desiderato di volte all'interno di un altro file. I file a cui si fa riferimento sono in formato *.txt* e non sono presenti all'interno del repository, ma sono facilmente reperibili.

2.2. Normalizzazione

Una volta estratto il testo questo viene sottoposto ad un processo di normalizzazione, in modo da ridurre la dimensione del dizionario complessivo e rendere più facile l'individuazione degli n-grams, questo viene fatto tramite le

funzioni `clean_chars` e `clean_words`. Nello specifico queste eliminano tutti i caratteri in maiuscolo, rimuovono la punteggiatura e nel caso dei caratteri eliminano tutti gli spazi fra parole diverse.

2.3. Counter

Come abbiamo visto lo scopo principale del progetto è contare le occorrenze degli n-grams, ed è importante notare come nel caso di testo dell'ordine dei GB è necessaria una struttura dati che ci permetta di memorizzare le istanze presenti nel dizionario in modo efficiente. Per questo all'interno del progetto viene usata la struttura dati `Counter`, questa è una Hash Map, che permette di accedere allo slot di memoria relativo ad un n-gram, senza dover scorrere la lista di tutti gli n-gram. Inoltre vedremo che questa struttura dati ci permetterà di sommare con semplicità i contatori parziali della versione parallela del codice.

3. Codice

In questa sezione andiamo a vedere le diverse versioni del codice che implementa il conteggio degli n-grams. I task principali sono due, ossia: il conteggio di n-gram di caratteri e il conteggio di n-gram di parole. Poichè il codice per svolgere i due task è molto simile, e varia principalmente nella fase di preprocessing, verrà mostrato solo quello riguardante i caratteri.

3.1. Sequential version

Iniziamo l'analisi con la versione sequenziale dell'algoritmo. Il codice per la versione sequenziale è contenuto nel file `sequential_counter` ed è formato da due fasi principali:

- Estrazione degli n-grams
- Conteggio delle occorrenze all'interno del testo

Di seguito vediamo le funzioni che svolgono questo compito, nel caso del conteggio dei bigrams formati da caratteri vicini.

```
def get_ngrams(seq, n):
    limite = len(seq) - n + 1
    for i in range(limite):
        sl = seq[i : i+n]
        yield tuple(sl)
```

Listing 1. Estrazione degli n-grams sequenziale.

```
def compute_bigrams(chars):
    b_chars = Counter(get_ngrams(chars, 2))
    return b_chars
```

Listing 2. Conteggio dei bi-grams

La prima funzione prende in input il testo che ha già subito la fase di preprocessing e la dimensione di n-grams desiderati (2 per i bi-grams, 3 per i tri-grams...). Poi scorrendo tutta la sequenza di testo estrae sottosequenze di caratteri lunghe quanto il parametro n, convertendole in tuple. Successivamente tramite il costrutto "yield" restituisce l'ngram generato alla funzione `get_ngrams` che si occupa di allocare la locazione di memoria che permette di contare le occorrenze nel caso sia un nuova n-grams, mentre aggiornerà il contatore per quelli che sono già presenti nella struttura dati. L'uso del costrutto `yield` è stato cruciale per quanto riguarda l'ottimizzazione del codice. Una prima soluzione era stata quella di inserire ogni n-gram trovato all'interno di una lista e poi passarla al Counter per permettere di contare le occorrenze; questo presentava problemi nel caso di corpus di dimensioni molto elevate. L'operazione di inserimento costringe a dover scorrere tutta la lista e quando il numero di n-grams è elevato, l'operazione diventa molto costosa. In questo modo invece ad ogni ciclo il codice si interrompe per passare l'n-gram alla funzione che ne andrà a fare l'hashing. Il codice sequenziale pertanto risulta semplice, ma riesce a svolgere il task senza problemi.

3.2. Parallel version

Dopo aver implementato la versione sequenziale del codice vediamo adesso come è possibile passare ad una versione parallela. La versione parallela è stata sviluppata mediante due librerie diverse che abbiamo visto durante le lezioni del corso:

- Multiprocessing: La libreria nativa di Python, permette di aggirare il GIL, andando a creare dei processi figli del programma originale.
- JobLib: Una libreria che sfrutta il backend "Loky", che permette di gestire i cali di memoria e prevenire i deadlocks. Inoltre permette di salvare grandi quantità di dati su una memoria condivisa permettendo ai processi di accedervi, evitando di clonare i dati.

Iniziamo l'analisi dal paredigma utilizato per fare il codice, ossia MapReduce. In questo approcco si divide il task in due fasi primitive denominate Map e Reduce applicate a coppie chiave-valore.Nella fase di Map, una funzione di mapping viene applicata in parallelo ad ogni split del dataset, dando in output delle coppie chiave-valore. Successivamente le coppie intermedie vengono raggruppate per chiave. Infine una funzione Reduce aggrega i risultati intermedi. Nel caso del conteggio degli n-grams la fase di Map individua gli n-grams presenti in ogni split del dataset, successivamente si procederà al conteggio parziale e infine la funzione Reduce andrà a sommare i risultati parziali. Il nostro obiettivo quindi è suddividere il testo totale in chunk della stessa grandezza, assegnare ogni chunk ad un core, far contare le

occorrenze degli n-grams ad ogni core nel proprio chunk ed infine sommare tutti i risultati ottenuti da ogni core.

3.2.1 Multiprocessing

Il primo metodo usato per sviluppare la versione parallela è attraverso l'utilizzo di *Multiprocessing*. Si noti che per tutte le versioni parallele il metodo `get_ngrams` resta uguale alla versione sequenziale e lo stesso vale per le funzioni che calcolano bigrams e trigrams. Questo succede perchè il nostro approccio consiste nel dividere il testo in molteplici chunk e applicare gli stessi metodi su questi, invece che sull'intero testo. E' necessaria pertanto delle funzioni che si occupino di queste operazioni. In particolare della creazione dei chunk e dell'assegnazione dei processi ad ogni core. Andiamo a vedere innanzitutto la funzione che crea i chunk.

```
def get_chunks(cleaned_text, num_chunks,
    max_ngram_size=3):
    chunk_size = len(cleaned_text) //
        num_chunks
    chunks = []
    overlap = max_ngram_size - 1

    for i in range(num_chunks):
        start = i * chunk_size
        if i == num_chunks - 1:
            end = len(cleaned_text)
        else:
            end = (i + 1) * chunk_size +
                overlap
        sl = cleaned_text[start:end]
        chunks.append(sl)

    return chunks
```

Listing 3. Creazione di chunk di testo.

Il codice andrà quindi a creare dei chunk del testo completo di dimensione uguale, che andranno divisi fra i vari core. Un aspetto importante riguarda la gestione dell'overlap fra i diversi chunk. Infatti dividendo in modo netto un chunk da un altro si andranno a perdere gli n-gram i cui caratteri si trovano alcuni in un chunk e altri nel successivo. Per risolvere il problema è stato aggiunto un parametro overlap che andrà a considerare nel caso in cui ci trovassimo alla fine di un chunk anche i successivi caratteri.

Adesso soffermiamoci sulla porzione di codice che implementa la divisione del task fra i vari core e che fa uso di *Multiprocessing*. In particolare forniamo la versione dei bigram di caratteri, il codice delle altre versioni si trova su Github ma è analogo a questo.

```
def compute_bigrams_parallel(chars,
    num_cores):
    chunks_b = get_chunks(chars, num_chunks
        =num_cores, max_ngram_size=2)
    with multiprocessing.Pool(processes=
        num_cores) as pool:
        partial_counters_b = pool.map(
            process_chunk_bigrams, chunks_b)
    return sum(partial_counters_b, Counter
        ())
```

Listing 4. Calcolo degi n-grams con Multiprocessing.

Il codice è suddiviso in due parti principali, in linea con il paradigma Map-Reduce. Nella prima parte il file viene diviso in un numero di chunk pari al numero di cores che si vogliono usare, successivamente vengono creati *numcores* processi separati e tramite la funzione `map` viene assegnato ad ogni core un chunk e viene applicata la funzione che conta le occorrenze degli n-grams. Nella seconda parte, viene fatta la fase di Reduce, dove si andranno a sommare i contatori parziali di ogni core.

3.2.2 JobLib

Come altro metodo abbiamo implementato il task tramite JobLib, anche in questo caso sia per gli n-gram di caratteri che per quelli di parole. Lo scopo di questo esperimento è vedere se tramite esperimenti con dimensioni di file più grandi si notano variazioni nelle performance fra metodi diversi di parallelizzazione. In questo caso la funzione per ottenere i chunk non subisce alcuna modifica rispetto a quella illustrata prima. Andiamo quindi a vedere come avviene il conteggio degli n-grams.

```
def compute_bigrams_joblib(chars, num_cores
    ):
    chunks_b = get_chunks(chars, num_chunks
        =num_cores, max_ngram_size=2)
    partial_counters_b = Parallel(n_jobs=
        num_cores, batch_size='auto')(
        delayed(process_chunk_bigrams)(
            chunk) for chunk in chunks_b
    )
    return sum(partial_counters_b, Counter
        ())
```

Listing 5. Calcolo degi n-grams con JobLib.

Come possiamo vedere il codice scritto sfruttando *JobLib* presenta una sintassi diversa rispetto a quello scritto con *Multiprocessing*. La parte principale del codice è quella usata per costruire il contatore parziale, ossia *Parallel*. A questa vengono indicati il numero di core e tramite il parametro *batchsize* impostato ad *auto* indichiamo l'ottimizzazione intelligente del carico, che vedremo risultare molto utile successivamente. Nelle parentesi seguenti viene indicato il modo con cui eseguire l'esecuzione, tramite il costrutto *delayed*; questo permette di non eseguire immediatamente l'esecuzione, in modo da attendere che i processi vengano assegnati ai core.

3.2.3 Threading

L'ultimo metodo da analizzare è tramite *Threading*, questo ho deciso di posizionarlo nel capitolo dedicato alle versioni parallele anche se vedremo che si differenzierà dagli altri due metodi. Le differenze principali sono che lo spazio di memoria fra i threads è condiviso, ma soprattutto è presente il meccanismo di context switching che permette ad un solo thread alla volta di svolgere il task.

4. Esperimenti

Iniziamo adesso la sezione del report che riguarda gli esperimenti effettuati, abbiamo visto come i tre diversi tipi di implementazione variano, adesso vediamo quanto queste variano a livello di prestazioni. Gli esperimenti sono stati fatti sui seguenti testi:

- Frankestein - Mary Shelley (78099 parole)
- Frankestein - Mary Shelley ripetuto per 50 volte (3.904.950 parole)
- Dataset "text8" (100 MB - 17.057.642 parole)

4.1. N-Gram di Caratteri

Valutiamo innanzitutto il calcolo delle occorrenze di n-gram di caratteri all'interno dei testi. Come primo esperimento valutiamo le quattro versioni in termini di tempo di esecuzione e speedup nel caso in cui nelle versioni parallele si vadano a creare chunk tutti della stessa dimensione, ossia del numero di caratteri totali diviso il numero di core. Ed usiamo un numero di core pari a 8 in tutte le versioni.

Tabella 1. Confronto prestazioni nel calcolo dei Bi-gram di caratteri (8 Core)

	Frankenstein		**Frank. x50**		**Text8 (100MB)**	
Metodo	Time (s)	Speedup	Time (s)	Speedup	Time (s)	Speedup
Sequenziale	0.12	-	5.32	-	25.02	-
Threading	0.10	1.2x	7.09	0.75x	27.78	0.90x
Multiprocessing	0.22	0.54x	2.05	2.59x	11.74	2.13x
JobLib	0.24	0.50x	2.24	2.37x	11.92	2.09x

Tabella 2. Confronto prestazioni nel calcolo dei Tri-gram di caratteri (8 Core)

	Frankenstein		**Frank. x50**		**Text8 (100MB)**	
Metodo	Time (s)	Speedup	Time (s)	Speedup	Time (s)	Speedup
Sequenziale	0.15	-	5.85	-	32.54	-
Threading	0.14	1.07x	7.22	0.81x	33.06	0.98x
Multiprocessing	0.18	0.83x	1.92	3.04x	16.89	1.92x
JobLib	0.17	0.88x	1.86	3,15x	16.47	1.97x

Come è possibile vedere dalle tabelle nel caso in cui viene fatto l'n-gram di caratteri, le prestazioni di entrambe le versioni parallele risultano migliori di quella sequenziale per quanto riguarda i testi lunghi. Per il solo testo di "Frankestein" le performance sono quasi uguali come tempo di esecuzione, con un leggero vantaggio della versione sequenziale. In questi casi il tempo impiegato per le operazioni di preprocessing (chunking e assegnazione) è paragonabile al tempo necessario ad eseguire il task, rendendo la parallelizzazione superflua. Possiamo notare come il task riguardante i trigram ottenga benefici migliori dalla parallelizzazione sul dataset *Frankestein x50*, ma vada a perdere su *Text8*, questo a prima vista può sembrare un comportamento anomalo, di cui ci concentreremo nelle prossime sezioni E' interessante invece notare nei testi di grande dimensione che lo speedup fra le due versioni parallele dello stesso task non varia molto, e diversamente da quanto era possibile pensare all'aumentare della dimensione del dataset lo speedup non migliora; anche questo verrà analizzato in seguito. Per quanto riguarda la versione con *Threading*, questa risulta essere al pari di quella sequenziale nel dataset più piccolo, ma peggiora drasticamente le performance con l'aumentare della dimensione.

4.2. N-Gram di Parole

Procediamo ora con l'analisi degli esperimenti riguardanti gli n-gram di parole. Anche in questo caso ripetiamo gli stessi esperimenti di prima, non andando a variare il numero di core.

Tabella 3. Confronto prestazioni nel calcolo dei Bi-gram di parole (8 Core)

	Frankenstein		**Frank. x50**		**Text8 (100MB)**	
Metodo	Time (s)	Speedup	Time (s)	Speedup	Time (s)	Speedup
Sequenziale	0.02	-	2.43	-	14.66	-
Threading	0.09	0.22x	2.38	1.02x	19.22	0.76x
Multiprocessing	0.35	0.05x	1.59	1.52x	33.10	0.44x
JobLib	0.39	0.05x	1.62	1.5x	31.47	0.47x

Tabella 4. Confronto prestazioni nel calcolo dei Tri-gram di parole (8 Core)

	Frankenstein		**Frank. x50**		**Text8 (100MB)**	
Metodo	Time (s)	Speedup	Time (s)	Speedup	Time (s)	Speedup
Sequenziale	0.03	-	2.66	-	15.22	-
Threading	0.10	0.30x	2.61	1.01x	20.57	0.73x
Multiprocessing	0.37	0.08x	1.78	1.49x	109.72	0.14x
JobLib	0.14	0.21x	1.83	1.45x	112.03	0.13x

Le tabelle mostrano risultati interessanti e piuttosto inaspettati, differentemente da prima gli approcci paralleli non scalano con la dimensione del file, ma dipendono dalla dimensione del vocabolario. Su testi piccoli (Frankestein) non ci aspettavamo grandi differenze, poichè il tempo di esecuzione dell'algoritmo è molto breve. Nel dataset medio i due approcci paralleli superano quello sequenziale ottenendo uno speedup di circa 1.5x in entrambi i casi. Il caso più eclatante è quello che riguarda il dataset *text8*. In questo scenario i metodi paralleli collassano impiegando più del doppio del tempo sui bi-gram rispetto alla versione sequenziale e quasi sette volte il tempo sui trigrammi. Questo è dovuto a due motivi principali: in primo luogo il dataset contiene un vocabolario vastissimo, che rende il numero di

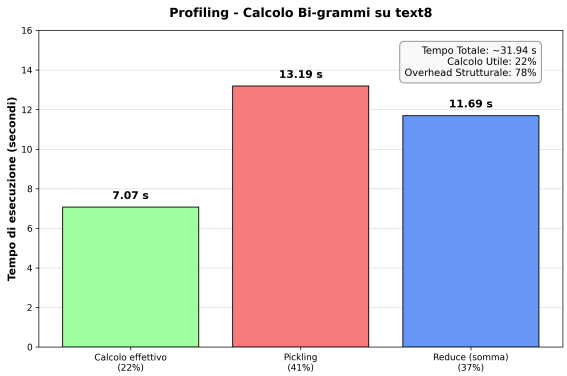

Figura 2. Profiling nel calcolo dei bi-gram di parole nel dataset *text8*.

n-gram totali nell'ordine dei milioni. I processi al termine del calcolo devono inviare i risultati sotto forma di dizionario al processo principale, saturando le risorse del sistema operativo. Il secondo motivo è da ritrovarsi nella parte che opera il reduce; la fase di somma infatti costringe a effettuare continui cicli di allocazione e deallocazione nella RAM. Il tutto senza dubbio è confermato dall'incremento repentino fra bi-grammi e tri-grammi. Tutto questo è confermato dal grafico in Figura 2 estratto da una fase di profiling effettuata sul codice che estrae i bi-grams di parole.

Come è possibile vedere il calcolo effettivo, ovvero il conteggio del numero di occorrenze effettuato da ogni processo, impiega solo 7.07 secondi, che rappresenta il 22% del tempo totale. Da notare che le inefficienze riscontrate con gli n-grams di parole svaniscono nel caso dei caratteri. Questo perchè, mentre il vocabolario di parole può crescere indefinitamente, il vocabolario dei caratteri è finito e di dimensione ristretta, infatti il numero di bigrammi massimo sarà $26^2 = 676$ e quello dei trigrammi $26^3 = 17576$. Questo dimostra chiaramente che per task memory-bound i costi dei trasferimenti dati in memoria annullano totalmente i benefici ottenuti dalla parallelizzazione.

4.3. Scalabilità

Valutiamo adesso le versione *Multiprocessing*, sullo stesso task svolto in precedenza, ma stavolta all'aumentare del numero di core utilizzati. Questo ci permetterà di valutare la configurazione ottimale con cui andare a fare il prossimo esperimento. Andiamo quindi a mostrare tempi di esecuzione e speedup ottenuto sul dataset *Frankestein-x50*

Possiamo notare nel grafico come per *Multiprocessing* le migliori performance si ottengano con 8 core sia nel caso dei bi-gram, che in quello dei tri-gram. Infatti se aumentiamo ulteriormente i valori di questo parametro, otteniamo risultati che non migliorano, raggiungendo il limite teorico imposto dalla Legge di Amdahl.

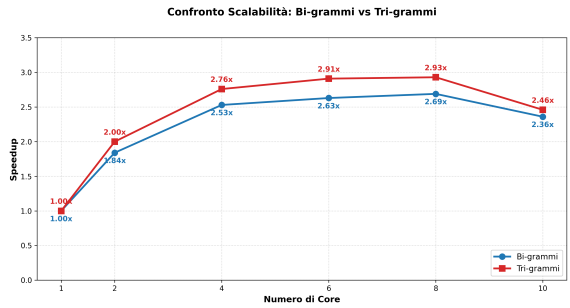

Figura 3. Speed-up al variare del numero di core sul Dataset *Frankestein-x50* .

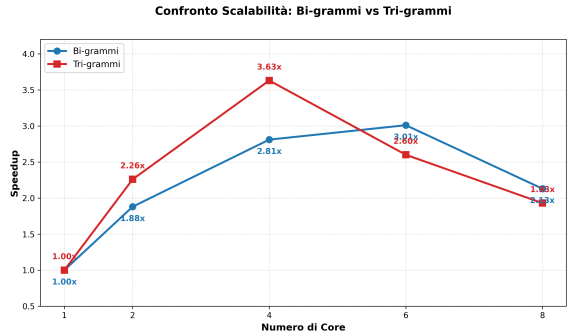

Figura 4. Speed-up al variare del numero di core sul Dataset *text8* .

E' interessante anche il confronto della scalabilità sul dataset *text8* per quanto riguarda il conteggio degli n-gram di caratteri. In questo caso, infatti vediamo un andamento molto più marcato, con un picco nella prestazioni in corrispondenza di 4 core, e successivamente un decremento molto rapido, dovuto probabilmente alla difficoltà di suddividere il testo fra processsi diversi, ma soprattutto alla fase di reduce, che fa sommare dizionari di dimensioni molto più grandi rispetto al dataset *Frankenstein-x50*.

4.4. Stress Test Multiprocessing vs JobLib

Procediamo ora con un esperimento che avvicina l'esperimento a quanto viene fatto solitamente nell'ambito del Natural Language Processing. Finora infatti abbiamo diviso in chunk in modo che avessero tutti uguale dimensione; nella realtà invece, spesso il testo viene diviso per capitoli o paragrafi che possono avere lunghezze diverse. Per questo andiamo a modificare la funzione `get_chunks` in modo che suddivida i chunk del testo in modo asimmetrico. In particolare andremo a creare appositamente una situazione fortemente sbilanciata, dove i primi K chunk contengono l'80% del testo, mentre gli altri N-K contengono il restante 20%.

5. Risultati

6. Conclusioni

7. Specifiche della macchina

7.1. The ruler

7.2. Mathematics

Please number all of your sections and displayed equations. It is important for readers to be able to refer to any particular equation. Just because you didn't refer to it in the text doesn't mean some future reader might not need to refer to it. It is cumbersome to have to use circumlocutions like "the equation second from the top of page 3 column 1". (Note that the ruler will not be present in the final copy, so is not an alternative to equation numbers). All authors will benefit from reading Mermin's description of how to write mathematics: `http://www.cvpr.org/doc/mermin.pdf`.

7.3. Miscellaneous

Compare the following:

`$conf_a$`	$conf_a$
`$\mathit{conf}_a$`	conf_a

See The TEXbook, p165.

The space after *e.g.*, meaning "for example", should not be a sentence-ending space. So *e.g.* is correct, *e.g.* is not. The provided `\eg` macro takes care of this.

When citing a multi-author paper, you may save space by using "et alia", shortened to "*et al.*" (not "*et. al.*" as "*et*" is a complete word.) However, use it only when there are three or more authors. Thus, the following is correct: " Frobnication has been trendy lately. It was introduced by Alpher [3], and subsequently developed by Alpher and Fotheringham-Smythe [1], and Alpher *et al*. [2]."

This is incorrect: "... subsequently developed by Alpher *et al*. [1] ..." because reference [1] has just two authors. If you use the `\etal` macro provided, then you need not worry about double periods when used at the end of a sentence as in Alpher *et al*.

For this citation style, keep multiple citations in numerical (not chronological) order, so prefer [1, 3, 4] to [3, 1, 4].

8. Formatting your paper

All text must be in a two-column format. The total allowable width of the text area is $6\frac{7}{8}$ inches (17.5 cm) wide by $8\frac{7}{8}$ inches (22.54 cm) high. Columns are to be $3\frac{1}{4}$ inches (8.25 cm) wide, with a $\frac{5}{16}$ inch (0.8 cm) space between them. The main title (on the first page) should begin 1.0 inch (2.54 cm) from the top edge of the page. The second and following pages should begin 1.0 inch (2.54 cm) from the top edge. On all pages, the bottom margin should be 1-1/8 inches (2.86 cm) from the bottom edge of the page for 8.5×11-inch paper; for A4 paper, approximately 1-5/8 inches (4.13 cm) from the bottom edge of the page.

8.1. Margins and page numbering

All printed material, including text, illustrations, and charts, must be kept within a print area 6-7/8 inches (17.5 cm) wide by 8-7/8 inches (22.54 cm) high.

8.2. Type-style and fonts

Wherever Times is specified, Times Roman may also be used. If neither is available on your word processor, please use the font closest in appearance to Times to which you have access.

MAIN TITLE. Center the title 1-3/8 inches (3.49 cm) from the top edge of the first page. The title should be in Times 14-point, boldface type. Capitalize the first letter of nouns, pronouns, verbs, adjectives, and adverbs; do not capitalize articles, coordinate conjunctions, or prepositions (unless the title begins with such a word). Leave two blank lines after the title.

AUTHOR NAME(s) and AFFILIATION(s) are to be centered beneath the title and printed in Times 12-point, non-boldface type. This information is to be followed by two blank lines.

The ABSTRACT and MAIN TEXT are to be in a two-column format.

MAIN TEXT. Type main text in 10-point Times, single-spaced. Do NOT use double-spacing. All paragraphs should be indented 1 pica (approx. 1/6 inch or 0.422 cm). Make sure your text is fully justified—that is, flush left and flush right. Please do not place any additional blank lines between paragraphs.

Figure and table captions should be 9-point Roman type as in Figures **??** and 5. Short captions should be centred.

Callouts should be 9-point Helvetica, non-boldface type. Initially capitalize only the first word of section titles and first-, second-, and third-order headings.

FIRST-ORDER HEADINGS. (For example, **1. Introduction**) should be Times 12-point boldface, initially capitalized, flush left, with one blank line before, and one blank line after.

SECOND-ORDER HEADINGS. (For example, **1.1. Database elements**) should be Times 11-point boldface, initially capitalized, flush left, with one blank line before, and one after. If you require a third-order heading (we discourage it), use 10-point Times, boldface, initially capitalized, flush left, preceded by one blank line, followed by a period and your text on the same line.

Figura 5. Example of a short caption, which should be centered.

Method	Frobnability
Theirs	Frumpy
Yours	Frobbly
Ours	Makes one's heart Frob

Tabella 5. Results. Ours is better.

8.3. Footnotes

Please use footnotes[1] sparingly. Indeed, try to avoid footnotes altogether and include necessary peripheral observations in the text (within parentheses, if you prefer, as in this sentence). If you wish to use a footnote, place it at the bottom of the column on the page on which it is referenced. Use Times 8-point type, single-spaced.

8.4. References

List and number all bibliographical references in 9-point Times, single-spaced, at the end of your paper. When referenced in the text, enclose the citation number in square brackets, for example [4]. Where appropriate, include the name(s) of editors of referenced books.

8.5. Illustrations, graphs, and photographs

All graphics should be centered. Please ensure that any point you wish to make is resolvable in a printed copy of the paper. Resize fonts in figures to match the font in the body text, and choose line widths which render effectively in print. Many readers (and reviewers), even of an electronic copy, will choose to print your paper in order to read it. You cannot insist that they do otherwise, and therefore must not assume that they can zoom in to see tiny details on a graphic.

[1] This is what a footnote looks like. It often distracts the reader from the main flow of the argument.

When placing figures in LaTeX, it's almost always best to use `\includegraphics`, and to specify the figure width as a multiple of the line width as in the example below

```
\usepackage[dvips]{graphicx} ...
\includegraphics[width=0.8\linewidth]
                {myfile.eps}
```

8.6. Color

Color is valuable, and will be visible to readers of the electronic copy. However ensure that, when printed on a monochrome printer, no important information is lost by the conversion to grayscale.

Riferimenti bibliografici

[1] A. Alpher, , and J. P. N. Fotheringham-Smythe. Frobnication revisited. *Journal of Foo*, 13(1):234–778, 2003.

[2] A. Alpher, , J. P. N. Fotheringham-Smythe, and G. Gamow. Can a machine frobnicate? *Journal of Foo*, 14(1):234–778, 2004.

[3] A. Alpher. Frobnication. *Journal of Foo*, 12(1):234–778, 2002.

[4] Authors. The frobnicatable foo filter, 2006. ECCV06 submission ID 324. Supplied as additional material `eccv06.pdf`.

9. Appendix

If your course project is part of a larger project from another class or research lab, please fill in this section and clearly spell out the following items:

1. Explicitly explain what the computer vision components are in this course project;
2. Explicitly list out all of your own contributions in this project in terms of:
   (a) ideas

(b) formulations of algorithms

(c) software and coding

(d) designs of experiments

(e) analysis of experiments

3. Verify and confirm that you (and your partner currently taking CS231A) are the sole author(s) of the writeup. Please provide papers, theses, or other documents related to this project so that we can compare with your own writeup.

Configurazione (4 Core)	**Frankenstein**		**Frank. x50**		**Dataset Text8**	
	Tempo (s)	Speedup	Tempo (s)	Speedup	Tempo (s)	Speedup
Sequenziale	0.12	-	5.85	1.00x	22.41	1.00x
Multiprocessing	0.15	-	1.95	3.00x	7.22	3.10x
JobLib	0.18	-	2.10	2.78x	7.05	3.17x

Tabella 6. Analisi comparativa dei tempi di esecuzione e speedup relativo.